

CSE552 Program Analysis

Assignment 1: Type Analysis

1 Overview

In this assignment, you will implement a type analysis for a subset of Rust.

Input. A Rust program where:

- every function parameter annotation is `()`,
- every function return type annotation is omitted,
- every local variable type annotation is omitted.

Output. If there exist type annotations that make the program well-typed, your analysis should find them. If no such annotations exist, your analysis should report failure.

2 Getting the Code and Running

2.1 Clone

```
git clone https://github.com/plrg-unist/cse552-a1.git
cd cse552-a1
```

2.2 Running and Testing

This assignment requires `rustup` (<https://rustup.rs/>).

```
cargo run
cargo run -- path/to/input.rs
cargo test
```

- `cargo run` reads an input Rust program, runs the type analysis, adds type annotations to the program according to the analysis result, and then type-checks the transformed program with the Rust compiler to validate that the annotations are correct. If the analysis reports failure, it prints `no solution`.
- `cargo run` reads from `stdin` if no file argument is given.
- `cargo test` runs the tests.

3 What You Need to Read

You only need these files:

- `src/analysis.rs`
- `src/types.rs`
- `src/union_find.rs`
- `src/tests.rs`

You do **not** need to read other files to complete the assignment.

Transformation is already done. `src/transformation.rs` is already implemented. It takes your analysis result and inserts type annotations into the program. You do **not** need to implement transformation yourself. You also do **not** need to read its implementation to finish this assignment (you may read it if interested).

4 What You May Change

- You are allowed to change **only** `src/analysis.rs`.
- You may remove/replace the template in `src/analysis.rs`.
 - The current template always succeeds (`Some(...)`) and assigns `()` to every local variable type and every function return type.
- You must preserve:
 - the `AnalysisResult` type definition,
 - the signature of `analyze`.
- The only submission file is `src/analysis.rs`.

Union-Find. `src/union_find.rs` provides a basic union-find implementation. You may use it directly, or implement your own union-find logic. If you implement your own, put it in `src/analysis.rs`.

5 Supported Rust Subset

Your implementation should handle a subset of Rust.

5.1 Types

The analysis works with:

- `bool`
- `i32`
- shared references `&T` (mutable references `&mut T` are not supported)

- tuples $(T_1, \dots, T_n)$
- function pointers $\text{fn}(T_1, \dots, T_n) \rightarrow R$
- type variables
- $\diamond$ (absent field)

5.2 Concrete Syntax and HIR

Concrete syntax (examples)	HIR variant
<code>fn f(x: ()) { ... }</code>	<code>ItemKind::Fn</code>
<code>let x = e;</code>	<code>StmtKind::Let</code>
<code>e;</code>	<code>StmtKind::Semi</code>
<code>e</code>	<code>StmtKind::Expr</code>
<code>f(e1, ..., en)</code>	<code>ExprKind::Call</code>
<code>(e1, ..., en)</code>	<code>ExprKind::Tup</code>
<code>e1 + e2</code>	<code>ExprKind::Binary</code>
<code>-e</code>	<code>ExprKind::Unary</code>
<code>1, true</code>	<code>ExprKind::Lit (Int, Bool)</code>
<code>e</code>	<code>ExprKind::DropTemps</code> (It is an internal HIR wrapper and does not appear as explicit source syntax.)
<code>if c { t }, if c { t } else { f }</code>	<code>ExprKind::If</code>
<code>loop { ... }</code>	<code>ExprKind::Loop</code>
<code>{ ... }</code>	<code>ExprKind::Block</code>
<code>{ ... }</code>	<code>Block</code>
<code>x = e</code>	<code>ExprKind::Assign</code>
<code>x += e</code>	<code>ExprKind::AssignOp</code>
<code>e.0</code>	<code>ExprKind::Field</code>
<code>x</code>	<code>ExprKind::Path</code>
<code>&e</code>	<code>ExprKind::AddrOf</code>
<code>break</code>	<code>ExprKind::Break</code>
<code>continue</code>	<code>ExprKind::Continue</code>
<code>return e</code>	<code>ExprKind::Ret</code>

Assumptions.

- The input program contains no free identifier.
- Local variables are always initialized at declaration time, so declarations have the form `let x = e;` and never `let x;`.
- For field projection, any tuple field name "i" is allowed, where i is a nonnegative integer.

- `!e` is restricted to `bool` operands only. (In real Rust, `!e` allows both boolean negation and integer bitwise-not cases.)
- `^`, `&` (bitwise AND), `|`, `==`, `!=`, `<`, `<=`, `>`, and `>=` are restricted to `i32` operands only. (In real Rust, these operators allow both integers and booleans.)
- The left-hand side of an assignment is a variable or a tuple field projection.
- The type of `loop` is always `()`. (In real Rust, the type of `loop` can be any type if it does not contain `break`.)
- `break` never carries a value (`break` only), while `return` may appear as either `return` or `return e`.

6 Testing and Grading

6.1 Test Helpers

In `src/tests.rs`, the grading logic corresponds to these helper patterns:

- `test(code)`: expects a successful solution, transforms code, then type-checks transformed code.
- `test_var(code)`: expects a successful and *general* solution that still contains type variables.
- `test_none(code)`: expects no solution (`None`).

6.2 Correctness Requirements

Your `analyze` implementation is correct only if:

1. If type annotations exist that make the program type-check, your analysis returns `Some(AnalysisResult)` and the transformed program passes Rust type checking.
2. If multiple valid solutions exist, your solution is general (keeps unconstrained parts as type variables), rather than over-constraining them.
3. If no valid typing exists, your analysis returns `None`.

6.3 Grading Policy

- Your score is proportional to **hidden** test pass rate.
- Late submission is prohibited.

6.4 Academic Integrity and LLM Policy

- Do not get help from other people for solving this assignment.
- Do not give your solution to other people.
- Do not upload your solution to any public repository.
- Any cheating results in an **F** grade.
- You are allowed to use LLM tools, and using them is highly recommended.

7 Recommendation

Add your own tests to `src/tests.rs`. This is strongly recommended because only a few tests are provided.